



Technical Documentation Hardware

Smart Cane 2025-2026

Weyers Axel · Bosvelt Cederic
Kennes Joyce · Gilles Dario
Taamouti Adnane · Doudouh Yassine

Institution: AP Hogeschool Antwerpen
Supervisors: De Vos Jeroen, Peeters Tom

Table of Contents

TERMS AND ABBREVIATIONS	4
1. PROJECT SUMMARY.....	5
2. TECHNICAL DESIGN	6
2.1 HIGH-LEVEL HARDWARE ARCHITECTURE.....	6
2.2 INFRASTRUCTURE IMPACT	7
2.2.1 <i>Hardware components</i>	7
2.2.2 <i>Software installed on the Raspberry Pi</i>	7
2.2.3 <i>Licences</i>	7
2.2.4 <i>Cloud environment</i>	7
2.2.5 <i>Scalability</i>	8
2.3 BILL OF MATERIALS (BOM).....	8
2.4 PROCESSING UNIT.....	9
2.5 SENSOR MODULES	9
2.5.1 <i>Ultrasonic sensor</i>	9
2.5.2 <i>Camera</i>	9
2.5.3 <i>Gyroscope / Accelerometer</i>	9
2.6 FEEDBACK ACTUATORS	10
2.6.1 <i>Audio</i>	10
2.6.2 <i>Vibration motor</i>	10
2.7 BUTTON LAYOUT	10
2.8 POWER SUPPLY	10
2.8.1 <i>Power source</i>	10
2.8.2 <i>UPS Module</i>	10
2.9 WIRING SCHEMATIC	11
2.9.1 <i>GPIO pinout overview</i>	11
2.9.2 <i>PCB Layout</i>	12
3. FIRMWARE MODULES.....	14
3.1 <i>BLE GATT Server (bluezero)</i>	14
3.2 <i>Obstacle detection module</i>	17
3.3 <i>AI object recognition</i>	18
3.4 <i>Button handler module</i>	19
3.5 <i>Boot sequence</i>	20
4. RELEASE PLAN	21
5. EXTERNAL SYSTEM INTERFACES	21
6. DATA MIGRATION	22
7. SECURITY AND AUTHORISATION ROLES	22
8. DOCUMENTATION.....	22
REFERENCES.....	23

Terms and Abbreviations

The following table defines all terms and abbreviations used in this document:

Term / Abbreviation	Description
SBC	Single Board Computer
BLE	Bluetooth Low Energy
GATT	Generic Attribute Profile: BLE service / characteristic structure
TTS	Text-to-Speech
YOLO / YOLOv8n	You Only Look Once: real-time object detection model (nano variant)
GPIO	General Purpose Input / Output
I2C	Inter-Integrated Circuit: serial bus for sensor communication
D-Bus	Desktop Bus: IPC system in Linux, used by BlueZ
BlueZ	Official Linux Bluetooth protocol stack
bluezero	Python wrapper library for BlueZ
pico2wave	Offline text-to-speech engine (SVOX Pico)
pw-play	PipeWire audio playback utility
BMS	Battery Management System
DFD	Data Flow Diagram
PAM8403	Class-D audio amplifier IC
UPS	Uninterruptible Power Supply
V4L2	Video4Linux2: Linux camera driver framework
CI/CD	Continuous Integration / Continuous Deployment

1. Project Summary

The Smart Cane is an assistive navigation device designed to support visually impaired individuals in their daily mobility. Safe and independent mobility remains a major challenge , particularly in environments where uneven infrastructure, poorly maintained roads, and dense traffic increase everyday risk. While a traditional white cane is an essential mobility aid, it mainly supports detection at ground level and offers limited awareness of obstacles positioned higher up, such as hanging branches, market stalls, or parked vehicles.

The Smart Cane project was developed to address this gap by improving environmental awareness and providing more timely feedback during navigation. The system combines obstacle awareness, user feedback, object recognition, and communication with a mobile companion application to support safer and more informed mobility.

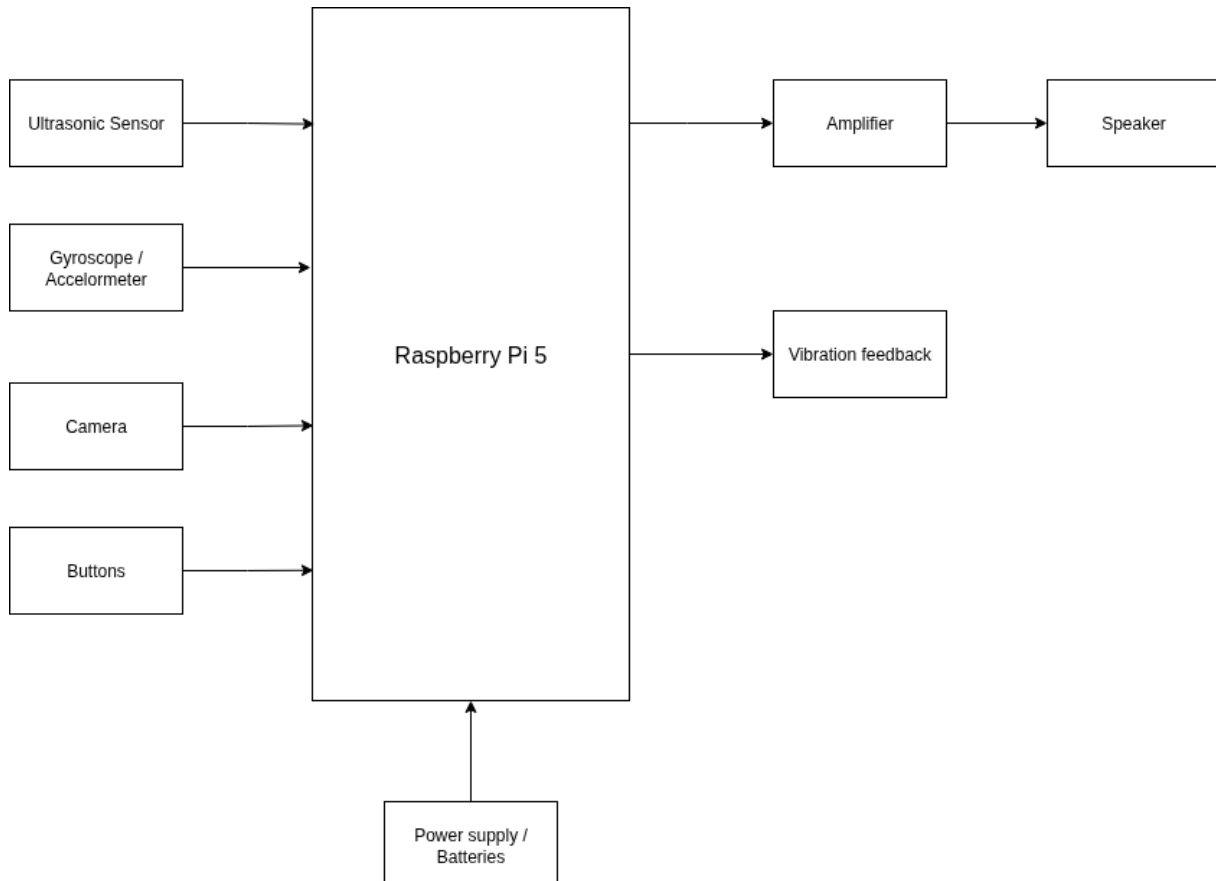
This document focuses specifically on the hardware design of the Smart Cane and describes the physical architecture, integration decisions, and technical building blocks that enable these functionalities. Detailed implementation aspects such as component selection, wiring, power design, and device-level integration are discussed in the following sections.



2. Technical design

2.1 High-level hardware architecture

The cane body integrates a Raspberry Pi 5 (8 GB) as its central processing unit, paired with a JSN-SR04T ultrasonic sensor for obstacle detection at hip-to-chest height and an MPU-6050 gyroscope/accelerometer for orientation measurement. A Waveshare OV5640 USB camera provides the input for on-demand AI object recognition.



Audio output for TTS and alerts is handled by a PAM8403 amplifier combined with a 3W speaker, while a DC vibration motor delivers haptic feedback whenever an obstacle is detected. 4 physical buttons: AI recognition, emergency, volume and distance adjustment. Power is supplied by a SupTronics X1202 UPS module paired with four 18650 Li-ion cells for battery management and USB-C charging. (In the prototype the power button is still on the raspberry PI, so you need to open the case to start or stop).

2.2 Infrastructure impact

2.2.1 Hardware components

The Smart Cane is a fully self-contained embedded system. All processing runs locally on the Raspberry Pi 5. The following physical infrastructure is required:

- Raspberry Pi 5 (8 GB)
- Sensors: JSN-SR04T (ultrasonic), MPU-6050 (IMU), Waveshare OV5640 USB camera
- Actuators: PAM8403 amplifier + 3W speaker, DC vibration motor
- Power: 4x 18650 Li-ion cells (3350 mAh each) in 4S configuration, SupTronics X1202 UPS module
- Physical enclosure: the cane body with secure mounting for all components

2.2.2 Software installed on the Raspberry Pi

The following software must be installed on Raspberry Pi OS on the device:

- Python 3 with: bluezero, gpiozero, ultralytics (YOLOv8n), smbus2, BlueZ Bluetooth stack
- pico2wave (SVOX Pico TTS engine)
- PipeWire audio system (pw-play)
- V4L2 camera drivers (included in Raspberry Pi OS)
- opencv-python (Apache 2.0)
- mpu6050-raspberrypi

Virtual environment: All Python dependencies are installed in a dedicated virtual environment created with `python -m venv env --system-site-packages`. The `--system-site-packages` flag is required so that system-level Bluetooth libraries (BlueZ bindings) are accessible from within the environment.

2.2.3 Licences

All software used on the Raspberry Pi is open-source and free of licensing cost:

- YOLOv8n (Ultralytics): AGPL-3.0 for open use
- BlueZ: GPL-2.0
- bluezero: Apache 2.0
- pico2wave (SVOX Pico): Apache 2.0
- gpiozero: BSD (Berkeley Software Distribution)
- Raspberry Pi OS: mixed open-source licences
- opencv-python: Apache 2.0
- mpu6050-raspberrypi: MIT
- smbus2: MIT (used by `gyro_sensor.py` via the mpu6050 library)

2.2.4 Cloud environment

The cane hardware itself requires no cloud environment. All AI inference (YOLOv8n), TTS generation (pico2wave), and sensor processing run entirely offline on the Raspberry Pi 5. The only network dependency is on the Android companion app side (Mapbox APIs, Google Play Services GPS), which is out of scope for this hardware document.

2.2.5 Scalability

The hardware design is for a single-device deployment. Each Smart Cane unit is an independent, self-contained device. Scaling to multiple units means assembling and flashing additional hardware, no shared infrastructure is required.

2.3 Bill of materials (BOM)

Name / Model	Category	Function	Price (EUR)
Raspberry Pi 5 (8 GB)	SBC / Main Controller	Central processing unit	82.60
Waveshare OV5640 5MP USB Camera (B)	USB Camera	Object recognition (YOLOv8n)	21.49
JSN-SR04T (waterproof)	Ultrasonic sensor	Obstacle detection up to ~5 m	7.44
SupTronics X1202 UPS Battery Module	UPS module	Battery management and USB-C charging	30.58
MPU-6050	Gyroscope / Accelerometer 3-Axis	Orientation measurement	6.61
Waveshare USB Sound Card + Amplifier 2.6 W	USB sound card / PAM8403	Audio amplification	7.02
3W 4 Ohm Speaker (compact)	Speaker	TTS voice instructions	1.65
DC Vibration Motor module 3.7-5.3 V	Vibration motor	Haptic obstacle feedback	2.07
Sanyo NCR18650GA 3350 mAh x4	Li-ion batteries	Primary power source	17.36
Total			176.82

2.4 Processing unit

The Raspberry Pi 5 was selected primarily because a unit was already available at school, eliminating any additional hardware cost for the prototype stage. This made it the most practical choice for development. However, the Raspberry Pi 5 is a full-sized SBC with a relatively large footprint. The intention for future production versions is to replace it with a more compact and power-efficient single-board computer or custom embedded module, allowing the overall form factor of the cane to be significantly reduced.

The Raspberry Pi 5, together with all electronics (UPS module, amplifier, and batteries), is housed in the large 3D-printed enclosure that forms the handle and body of the cane. The four physical buttons (VOL, DIST, AI, SOS) are mounted on the shaft section between the tip and the handle, positioned near where the user's thumb rests when gripping the cane.



2.5 Sensor modules

2.5.1 Ultrasonic sensor

The JSN-SR04T was selected for its waterproof housing, making it suitable for outdoor and wet-weather use. Its detection range is software-limited to approximately 5 metres, a threshold validated during field testing and is positioned to cover hip-to-chest height, the blind spot that a conventional white cane typically misses.

2.5.2 Camera

The OV5640 camera is integrated into the handle, pointing forward, and provides sufficient resolution (640×480) for reliable YOLOv8n object detection. The camera capture (cv2.VideoCapture) is opened at system startup and kept open throughout the session to reduce per-scan latency. Object detection itself is triggered on demand via a button press. It communicates over USB 2.0 using the standard V4L2 driver, requiring no custom drivers.

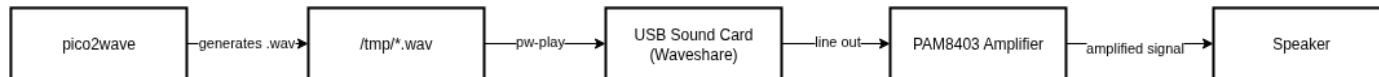
2.5.3 Gyroscope / Accelerometer

The gyroscope is used exclusively for fall detection, the GyroSensor class reads the accelerometer axes via the mpu6050 Python library and calls `check_fall()` in the main loop at approximately 10 Hz. If the magnitude of the acceleration vector deviates beyond a configured threshold, a fall event is flagged and an audio alert is played.

2.6 Feedback actuators

2.6.1 Audio

Audio feedback is provided by a Waveshare USB sound card with a built-in PAM8403 amplifier (2.6 W output), paired with a compact 3W 4 Ohm speaker directed towards the user. It handles TTS navigation instructions, obstacle warnings/messages.



Pico2wave is an offline text-to-speech engine based on the SVOX Pico TTS library. It converts a text string directly to a .wav audio file with no internet connection required.

2.6.2 Vibration motor

The DC vibration motor provides haptic feedback whenever an obstacle is detected within range, running continuously as long as the obstacle remains present. It is driven by a GPIO PWM output at a supply voltage of 3.7 to 5.3 V.

2.7 Button layout

The Smart Cane has 4 physical push buttons. All buttons require hardware debouncing or software debounce logic. The table below describes each button and its function:

Button	Normal Mode	Trigger
Emergency	Trigger SOS workflow	Hold 3 seconds
AI-recognition	Trigger AI-recognition workflow	Short press
Distance	Change detection threshold	Short press
Volume	Change audio volume	Short press

2.8 Power supply

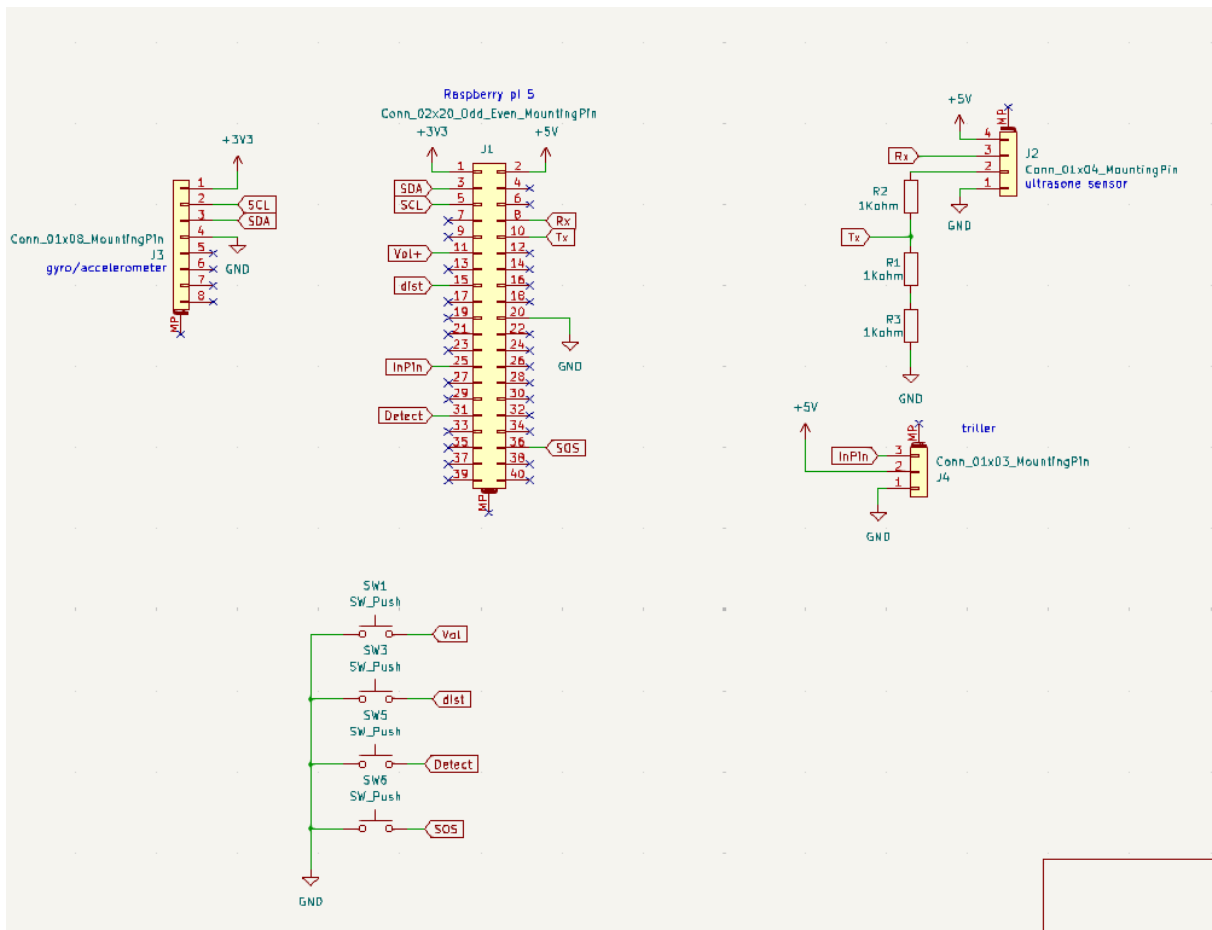
2.8.1 Power source

The Smart Cane is powered by a rechargeable battery pack consisting of four Sanyo NCR18650GA Li-ion cells. With a capacity of 3350 mAh per cell, the pack provides a total capacity of 13,400 mAh. The cells are connected in a 4S series configuration, resulting in a nominal output voltage of 14.8 V. Battery charging is managed via USB-C through the SupTronics X1202 UPS module, which is integrated into the device's power supply architecture.

2.8.2 UPS Module

SupTronics X1202 allows hot-swapping between battery and mains power without interrupting the Pi, Integrated battery management and charge control. Designed specifically for Raspberry Pi 5, USB-C input and output.

2.9 Wiring schematic



The schematic captures all major electrical connections between the Raspberry Pi 5 and its peripherals. The MPU-6050 gyroscope/accelerometer communicates over the I2C1 bus, with GPIO 2 (SDA) and GPIO 3 (SCL) operating at 3.3 V. All four push buttons are wired with pull-up or pull-down resistors to ensure clean, debounce-ready signal levels. The vibration motor is driven through a transistor or motor driver stage on a GPIO PWM pin, isolating the motor's current draw from the Pi. The USB sound card and USB camera connect directly to the Pi's USB-A ports and require no additional circuitry.

Because the ultrasonic sensor sends back 5 V to the raspberry pi, we had to place a voltage divider to weaken the voltage to 3.3 V.

2.9.1 GPIO pinout overview

The following table lists all GPIO connections:

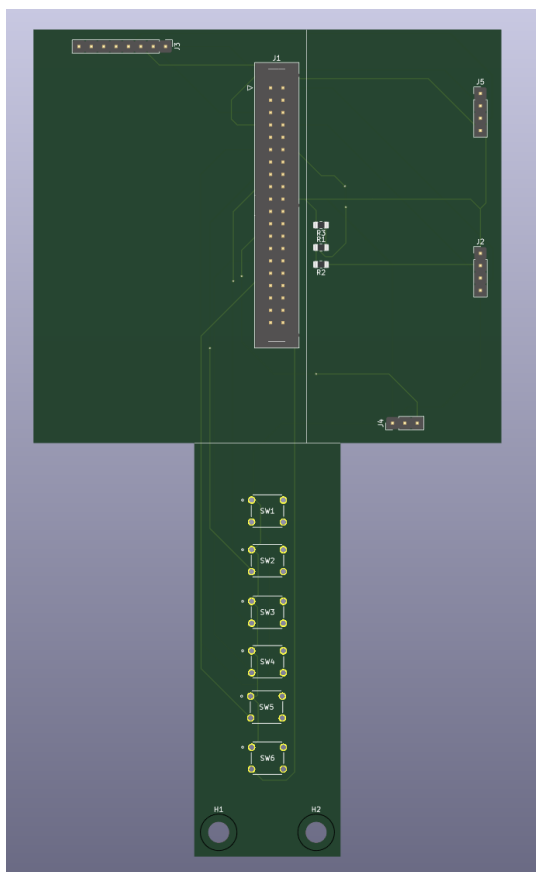
Component	Pi GPIO (BCM)	Physical Pin	Direction	Note
JSN-SR04T Trigger	GPIO 14	Pin 8	Output	Trigger pulse to sensor
JSN-SR04T Echo	GPIO 15	Pin 10	Input	Echo return from sensor
MPU-6050 SDA	GPIO 2 (I2C1 SDA)	Pin 3	I2C	3.3 V logic

MPU-6050 SCL	GPIO 3 (I2C1 SCL)	Pin 5	I2C	3.3 V logic
Vibration Motor	GPIO 4	Pin 7	Output / PWM	Via PWMOutputDevice
Emergency button	GPIO 27	Pin 13	Input	Pull-up, hold 3s triggers sos
Distance button	GPIO 5	Pin 29	Input	Pull-up
AI recognition button	GPIO 22	Pin 15	Input	Pull-up
Volume button	GPIO 17	Pin 11	Input	Pull-up

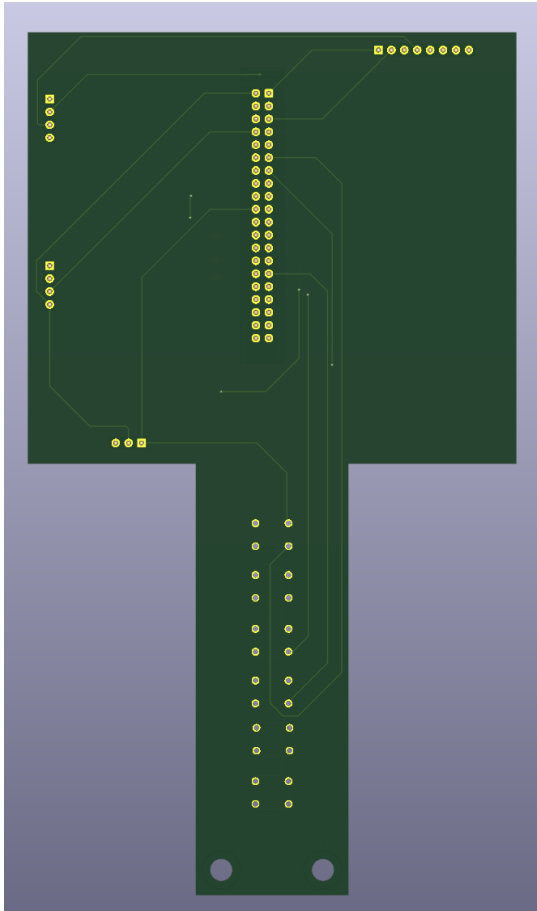
In the current prototype, the power button is the hardware button on the Raspberry Pi board itself. Opening the enclosure is required to power the device on or off. A dedicated GPIO-connected power button is planned for future hardware revisions.

2.9.2 PCB Layout

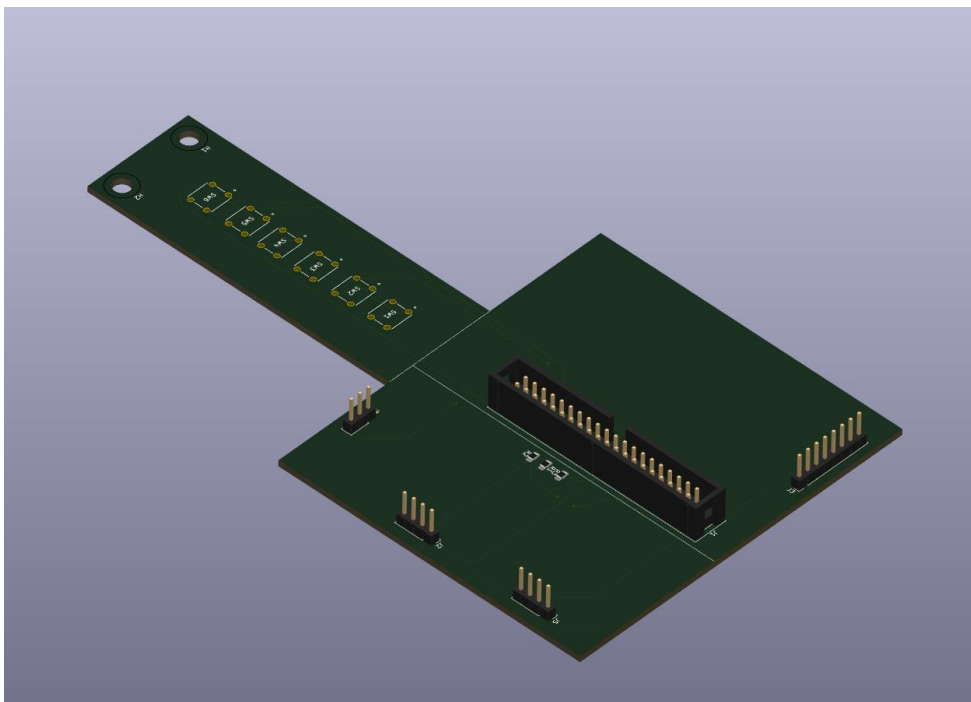
Top layer:



Bottomlayer:



Isometrical projection:



3. Firmware modules

3.1 BLE GATT Server (bluezero)

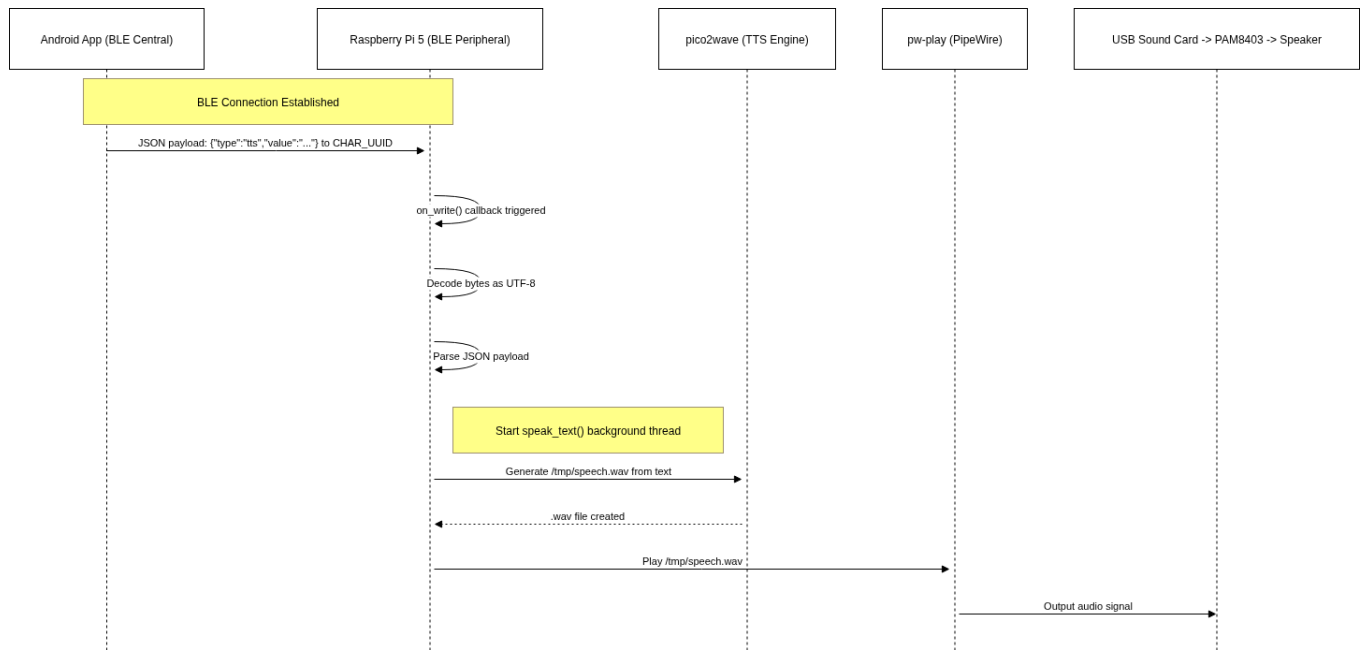
The main firmware process (main.py) and the Bluetooth server (bluetooth/bluetooth_server.py) run as two separate OS processes, started via subprocess.Popen. Direct inter-process communication is implemented through a temporary file: when the emergency button is held for 3 seconds, main.py writes "1" to /tmp/sos_trigger. The Bluetooth server polls this file every 500 ms in a background thread; when the file is detected, the server removes it and calls on_send(), which writes the value "SOS:TRIGGER" to the EMERGENCY characteristic, triggering a BLE NOTIFY to the connected Android app.

The Pi runs a BLE GATT peripheral implemented in Python 3 using the bluezero library, which communicates with the BlueZ Bluetooth stack via D-Bus IPC. Configuration:

Attribute	Value	Remark
GATT Role	Peripheral (Server)	Pi advertises; never initiates
BLE Local Name	SmartStick_BLE	Used by Android app for discovery
SERVICE_UUID	12345678-1234-5678-1234-567812345678	Project-specific custom UUID
CHAR_UUID	12345678-1234-5678-1234-567812345679	NAV characteristic, navigation text from Android to Pi
EMERGENCY_UUID	12345678-1234-5678-1234-56781234567A	SOS characteristic, Pi to Android
Bluetooth stack	BlueZ (Linux)	Used by bluezero via D-Bus IPC
Python library	bluezero	IPC via D-Bus

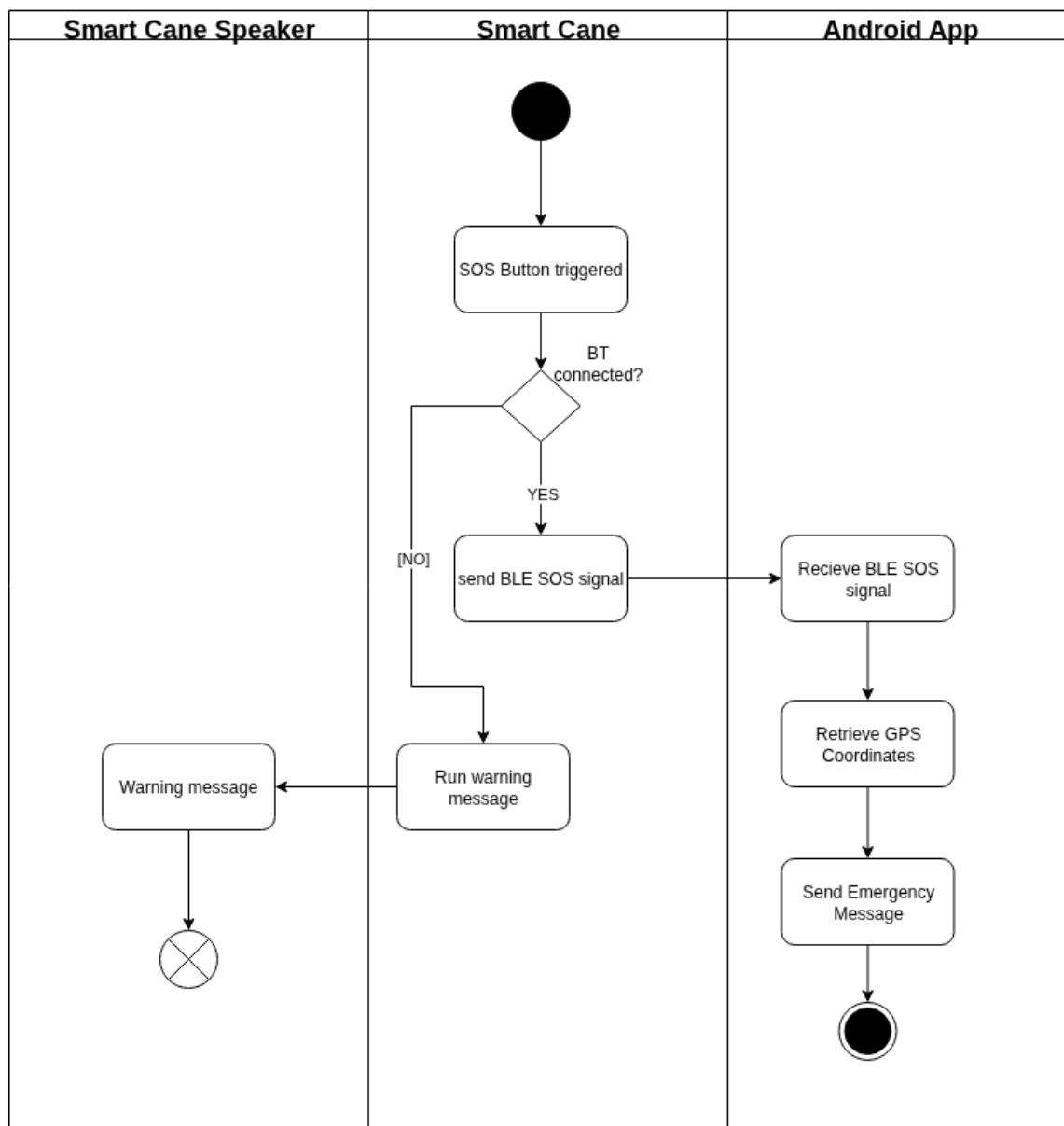
Bluetooth Low Energy communication is structured around the Generic Attribute Profile (GATT). A GATT server exposes services, each identified by a UUID, that a client can read, write, or subscribe to. The Smart Cane uses a project-specific custom service UUID (not a Bluetooth SIG standardised service) to avoid conflicts with standard profiles. The service exposes two characteristics: one for receiving TTS and volume commands from the app (write), and one for sending emergency notifications to the app (notify).

BLE data flow through the on_write() callback:



1. Android app connects to the Pi over BLE
2. App writes a JSON payload (UTF-8 encoded), with the structure `{"type": "tts", "value": "..."} for navigation text or {"type": "volume", "value": 80} for volume control`
3. `on_write()` decodes the raw bytes as UTF-8 and parses the JSON payload
4. For type: "tts": a background thread calls `speak_text()`, which runs `pico2wave` and `pw-play` to convert and play the text
5. For type: "volume": a debounced timer applies the volume level using `wpctl set-volume` via PipeWire, preventing flooding when a slider is used in the app

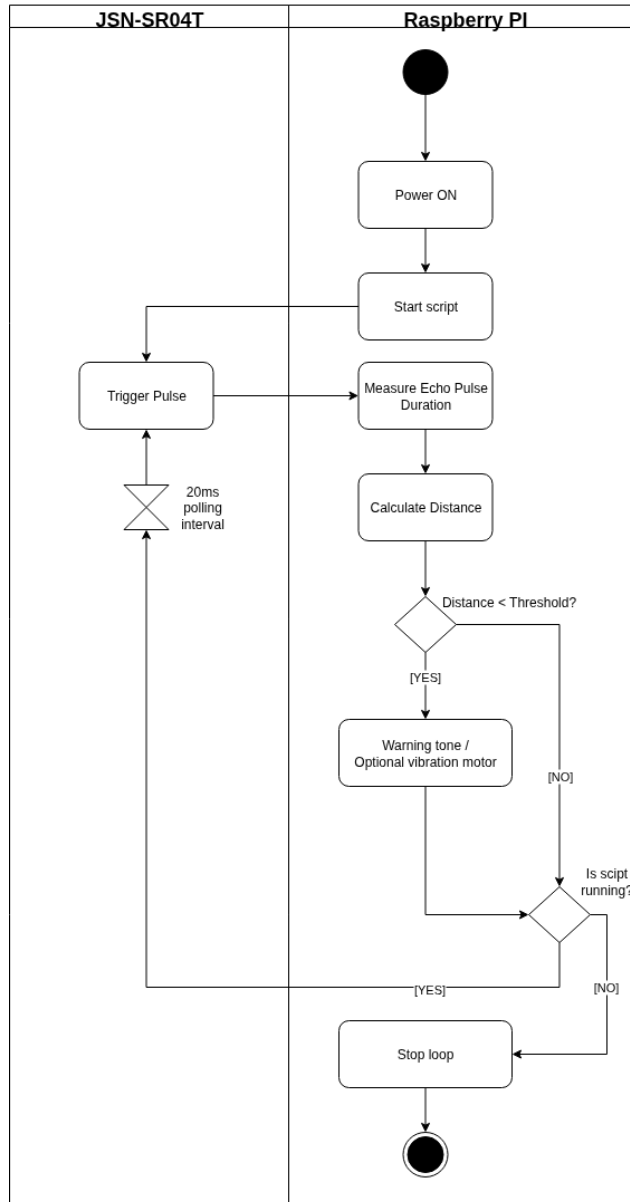
Emergency delivery mechanism:



On emergency button hold (3 s), the Pi writes to the EMERGENCY characteristic (NOTIFY), triggering the SOS workflow on the Android app. More information in the Software documentation of the Smart-cane.

3.2 Obstacle detection module

The system continuously polls the JSN-SR04T ultrasonic sensor using GPIO trigger/echo mode, identical to the HC-SR04 protocol. The Pi asserts a trigger pulse on GPIO 14 and measures the echo return time on GPIO 15, via the gpiozero DistanceSensor class. Users adjust the detection threshold with the Distance buttons. When an obstacle is within the threshold, the vibration motor activates and an audio warning tone is played.



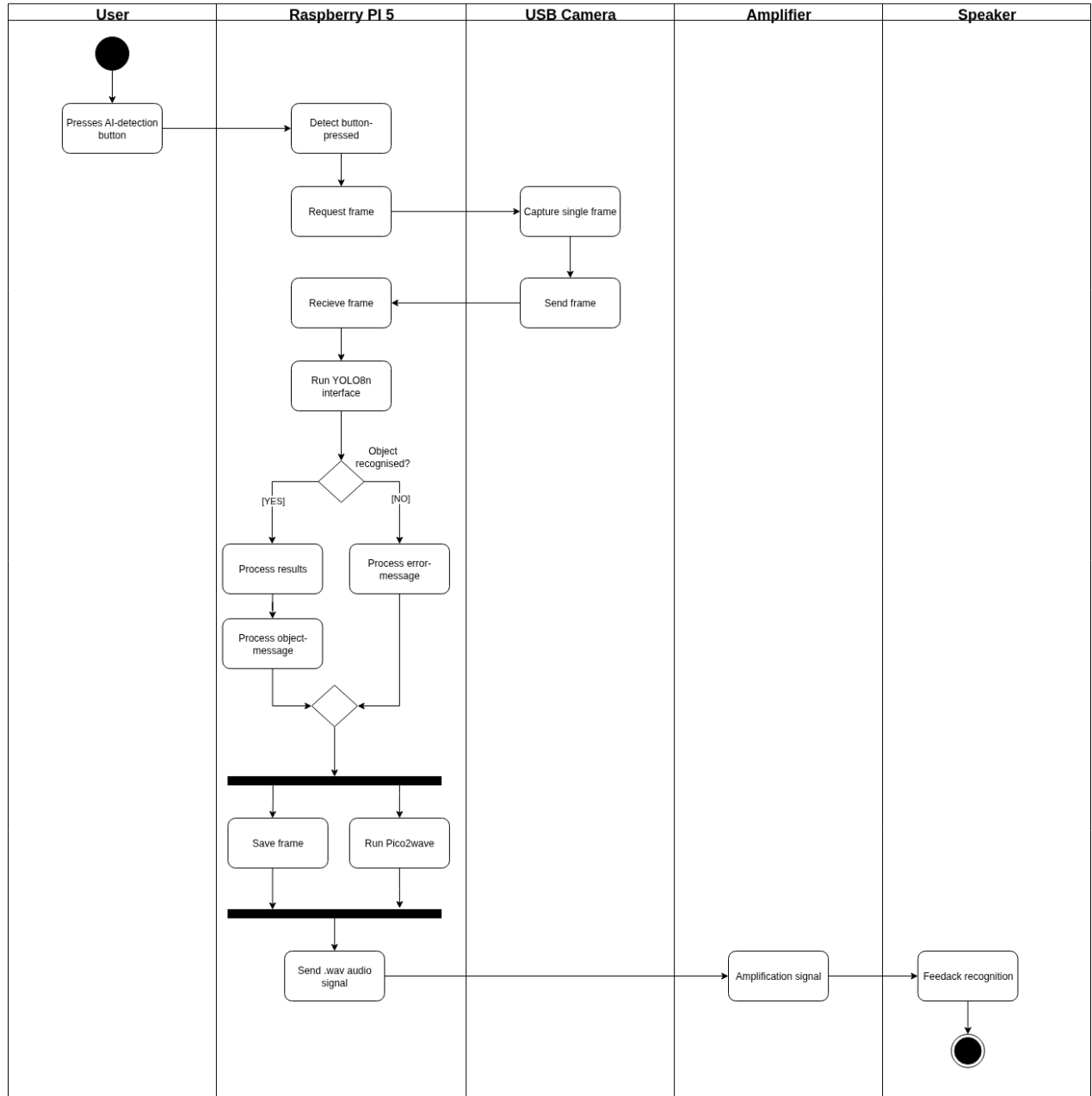
Parameter	Value
Polling Interval	20 ms
Detection Threshold (min / blind zone)	20 cm
Detection Threshold (default)	200 cm
Detection Threshold (medium)	350 cm
Detection Threshold (max)	500 cm

During development, the JSN-SR04T sensor produced highly variable readings in environments with mixed near and far surfaces. This is caused by the sensor's wide beam angle. It simultaneously detects both the nearby object and empty space further away, resulting in erratic distance values. This was resolved by applying a median filter: the last 5 raw readings are buffered, sorted, and the median value is used as the stable distance measurement. This significantly reduces false obstacle triggers.

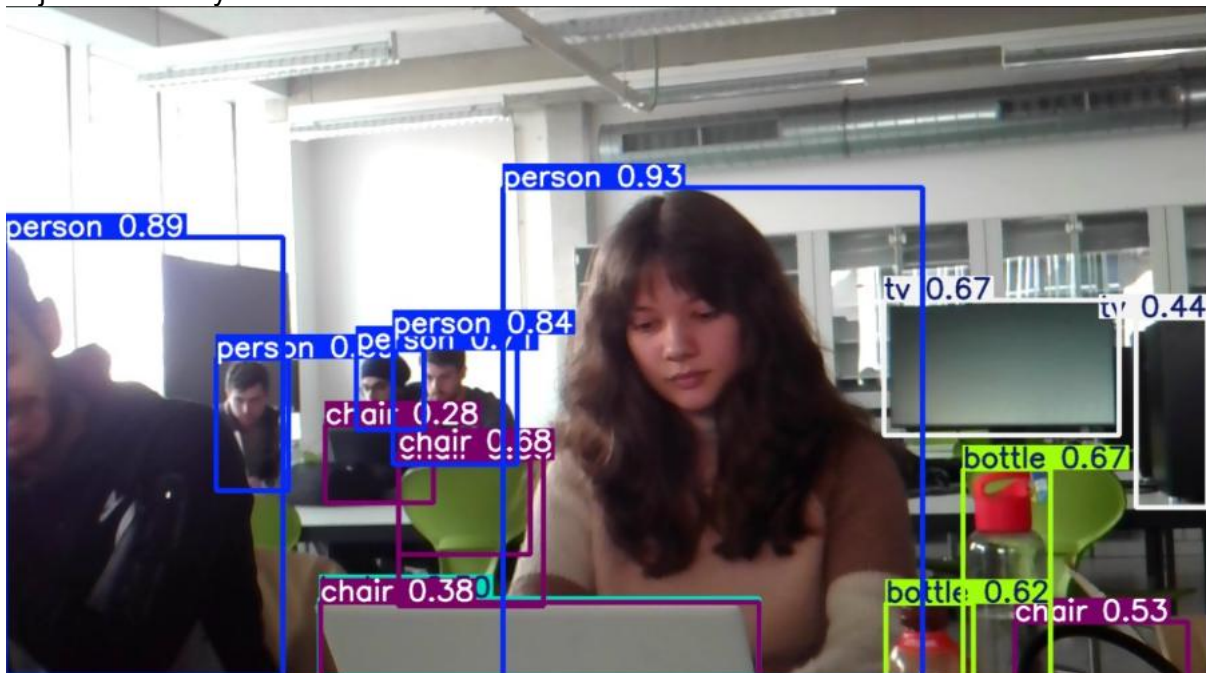
3.3 AI object recognition

When the AI recognition button is pressed, a single frame is captured from the USB camera via V4L2. The YOLOv8n nano model runs inference on the Raspberry Pi 5 CPU and returns either a list of detected objects or if no objects are recognised “clear”.

The activity diagram below illustrates the full flow from button press to audio output:



The image below shows an example of YOLOv8n object detection output, with each detected object labeled by class and confidence score.

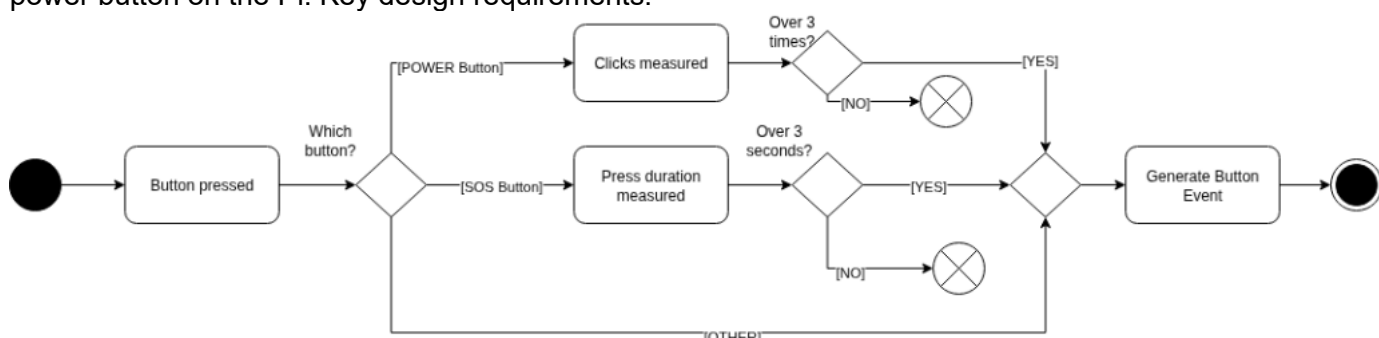


The annotated frame (with bounding boxes and confidence scores) is saved to disk as `latest_detection.jpg` in the project directory. This file is intentionally retained rather than deleted to allow developers and testers to inspect exactly what the AI model detected and at what confidence. It is overwritten on every scan.

Retaining the image is appropriate for a development prototype being tested in a controlled environment. In a production version intended for public use, the captured frame must be deleted immediately after inference to comply with applicable privacy regulations (including GDPR in Belgium/EU), as the camera may capture identifiable individuals in public spaces.

3.4 Button handler module

The button handler module implements a state machine for all 4 physical buttons and the power button on the Pi. Key design requirements:

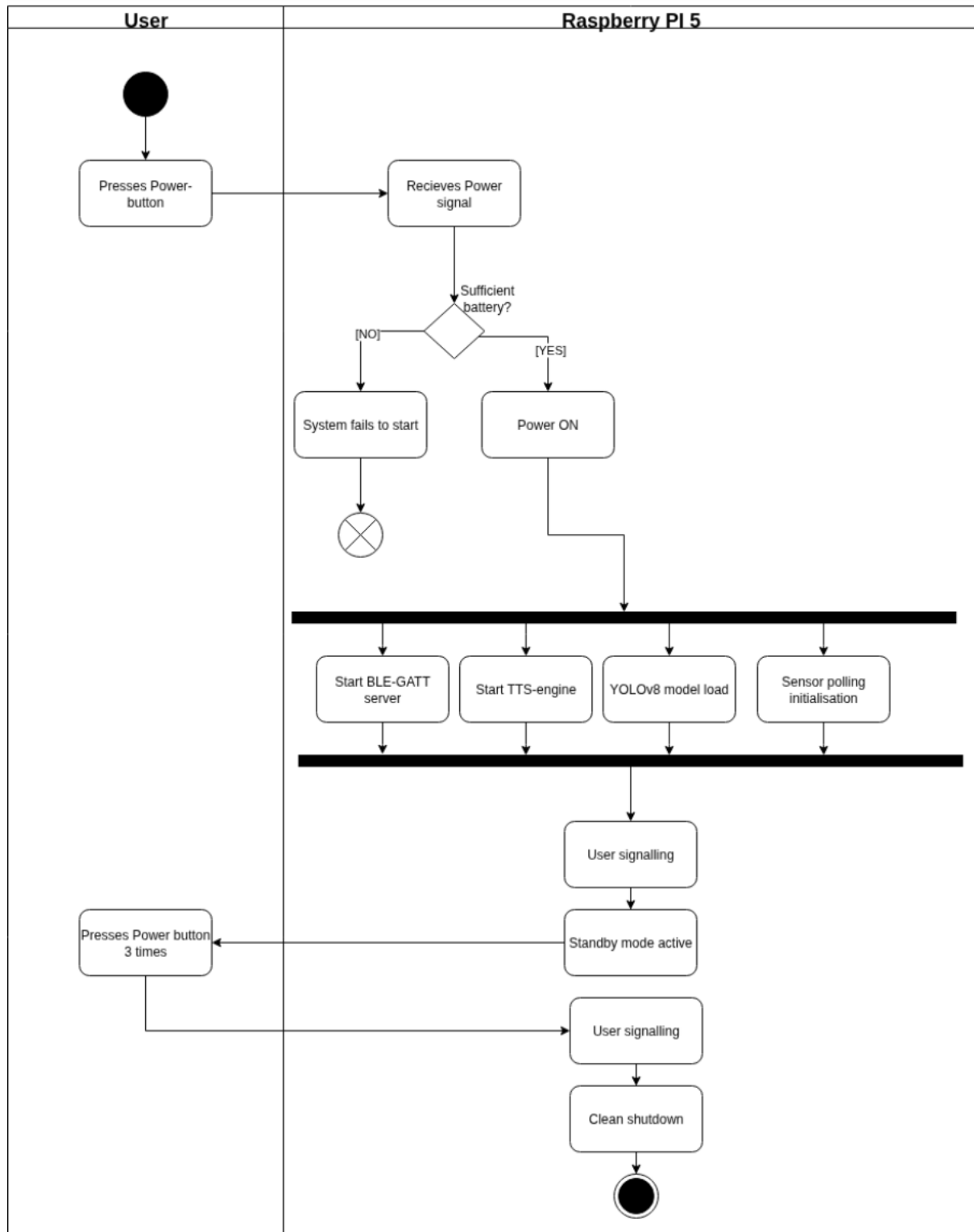


The button handler module uses interrupt-driven or polling-based GPIO detection for all buttons. Hardware or software debouncing is applied to ensure clean signal reads.

The power button turns the system on with a short press or off with three fast presses. The emergency button triggers the SOS workflow when held for three seconds, sending a BLE signal to the Android companion app. The distance button adjusts the obstacle detection threshold with each short press, and the volume button adjusts the audio playback volume in the same way.

3.5 Boot sequence

On power-on, the following initialisation sequence executes in parallel after the battery check passes:



1. Power button pressed => Pi receives power signal => battery check
2. Battery sufficient => Power ON; insufficient => system fails to start
3. Parallel initialisation: BLE-GATT server start, TTS engine start, YOLOv8n model load, sensor polling initialisation
4. User signalling (audio/vibration confirmation) => Standby mode active

5. Shutdown: Power button pressed 3 times => user signalling => clean shutdown

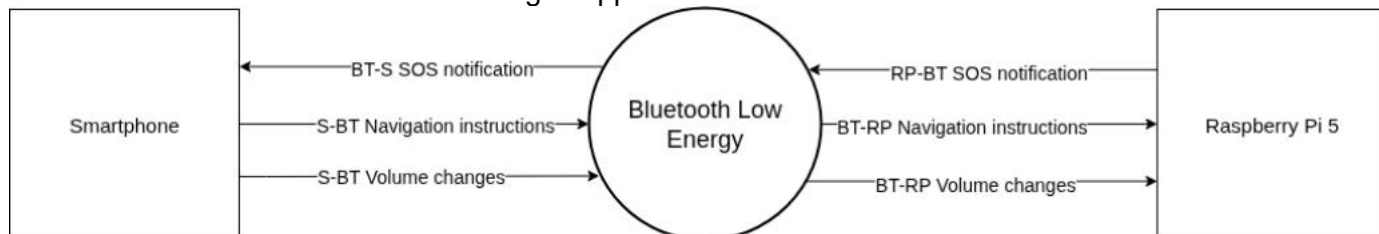
4. Release plan

The following sequence describes the full hardware build and bring-up process:

1. Assemble the cane body and mount the Raspberry Pi 5 and all electronics inside the 3D-printed handle enclosure.
2. Wire all sensors and actuators according to the KiCad schematic.
3. Connect the Waveshare USB sound card and the OV5640 USB camera to the Pi's USB-A ports.
4. Install the SupTronics X1202 UPS module and connect the 4x 18650 Li-ion cells. Verify BMS protection is active.
5. Flash Raspberry Pi OS (64-bit, latest stable) to a microSD card.
6. Install required Python packages: `python -m venv env --system-site-packages; source env/bin/activate; pip install ultralytics opencv-python gpiozero mpu6050-raspberrypi pyserial bluezero`
7. Install system packages: `sudo apt install pico2wave pipewire bluez`
8. Deploy the firmware scripts from the GitLab repository to the Pi.
9. Configure the main firmware script as a systemd service to start automatically on boot.
10. Perform functional verification: power on, BLE-advertisement visible, obstacle detection active, TTS playback confirmed.

5. External system interfaces

The Smart Cane communicates with external systems through its Bluetooth Low Energy interface to the Android companion application. In this communication model, the Raspberry Pi 5 acts as the BLE GATT peripheral, while the Android application acts as the BLE central that initiates the connection and exchanges application data with the cane.



The BLE-interface is used for two main communication flows.

First, the Android application sends navigation text instructions to the cane through a writable GATT characteristic, after which the Raspberry Pi converts the received text into speech and plays it through the speaker. Second, the cane can send an SOS-related trigger to the Android application when the emergency button is activated, after which the smartphone handles the emergency communication workflow.

The communication in this interface is based on Bluetooth Low Energy 5.0 and the Generic Attribute Profile (GATT). On the Raspberry Pi side, the BLE service is implemented through BlueZ and the bluezero Python library, which communicate through D-Bus.

6. Data migration

The Smart Cane is a self-contained embedded system with no persistent database or user data store on the device itself. No traditional data migration between environments is required. Firmware Deployment (Automated). New or updated Python firmware modules are deployed to the Raspberry Pi 5 from the GitLab repository.

The update process is: (1) Developer pushes a new commit to the main branch on GitLab. (2) The developer runs git pull on the Raspberry Pi to retrieve the latest firmware. (3) The pistick.service systemd unit is restarted (sudo systemctl restart pistick). (4) The system announces "System ready." via the speaker to confirm a successful boot. This process is fully manual in the current implementation. No configuration state (detection thresholds, volume level) is persisted to disk between reboots. All runtime settings reset to their defaults on each startup. Detection thresholds default to 200 cm and volume is read from the system PipeWire audio level at boot time.

7. Security and authorisation roles

The Smart Cane follows a single-user model. There are no login accounts, passwords, or role-based access control mechanisms on the device.

Physical User: full access to all cane functions via the four physical buttons. No authentication required for physical button interaction.

Android App User: must complete the BLE bonding procedure to connect. After bonding, the app can send TTS text instructions and volume commands over BLE. The app cannot access GPIO, sensor data, or the file system of the Raspberry Pi.

Developer: SSH access to the Raspberry Pi OS over a local network connection for firmware deployment, log inspection, and configuration changes. This access is protected by the standard Raspberry Pi OS user credentials and is not available wirelessly in the field.

8. Documentation

The firmware source code is documented through inline comments placed directly alongside the relevant logic. Non-obvious decisions, hardware-specific behaviour, and safety-critical operations are explained at the point where they occur in the code.

For a complete overview of the system, this document should be read alongside the Technical Design Software Document, which covers the Android companion application, and the User Guide Document, which serves as the end-user guide for operating both the Smart Cane and the companion app.

A CI/CD pipeline is documented separately in the CI/CD documentation. This pipeline applies exclusively to the Android companion application (automated build, test, and deployment). It has no impact on the hardware firmware or the Raspberry Pi deployment process.

References

The BlueZ Project. (2024). BlueZ: Official Linux Bluetooth protocol stack.

<http://www.bluez.org/>

Diodes Incorporated. (2015). PAM8403 3W, filterless, class-D stereo audio amplifier datasheet. <https://www.diodes.com/assets/Datasheets/PAM8403.pdf>

Elecrow. (2022). Ultrasonic ranging module JSN-SR04T.

https://www.elecrow.com/wiki/index.php?title=Ultrasonic_Ranging_Module_JSN-SR04T

Huckridge, D. (2024). python-bluezero: A Python module to allow simpler control of Bluetooth LE devices. GitHub. <https://github.com/ukBaz/python-bluezero>

InvenSense / TDK. (2013). MPU-6000 and MPU-6050 product specification revision 3.4.

<https://invensense.tdk.com/wp-content/uploads/2015/02/MPU-6000-Datasheet1.pdf>

naggety. (2011). picotts: Pico text to speech. GitHub. <https://github.com/naggety/picotts>

PipeWire contributors. (2024). PipeWire. <https://pipewire.org/>

Raspberry Pi Ltd. (2023). Raspberry Pi 5 product brief.

<https://datasheets.raspberrypi.com/rpi5/raspberry-pi-5-product-brief.pdf>

SupTronics. (2023). X1202 UPS power supply module for Raspberry Pi 5.

<https://www.suptronics.com/Xseries/x1202.html>

Ultralytics. (2023). YOLOv8 documentation. <https://docs.ultralytics.com/models/yolov8/>

Waveshare. (2023). OV5640 camera board (B) wiki.

[https://www.waveshare.com/wiki/OV5640_Camera_Board_\(B\)](https://www.waveshare.com/wiki/OV5640_Camera_Board_(B))